

Wochenprotokoll: VisionQuest

Name: Matthias

Datum: 20.02.2026

Meilenstein: M1

Projektstatus:  Im Plan /  Verzögert /  Kritisch

1. Was wurde diese Woche gemacht?

- Flutter Projektstruktur mit flutter create erstellt
- Express.js Backend Server mit CORS konfiguriert
- SQLite Datenbank Schema mit 4 Tabellen (users, quests, user_stats, achievements) definiert
- Package.json und pubspec.yaml mit allen Dependencies konfiguriert
- API-Struktur mit Route-Handling geplant
- Node.js Server läuft auf Port 5000

2. Wie liegt man im Zeitplan?

- M1 Deadline (27.02.2026) wird eingehalten
- Alle initialen Setup-Tasks abgeschlossen
- Basis-Infrastruktur fertig und funktionsfähig

3. Was sind die nächsten Schritte?

- CRUD-Operationen in dbService.js implementieren
- REST-API Endpoints definieren und testen
- JWT Authentication implementieren
- Flutter Frontend mit Login-Screen erweitern
- Error-Handling für API-Calls implementieren

4. KI-Einsatz & Dokumentation

Modell: GitHub Copilot (GPT-4)

1. Projekt-Befehle und Setup

Copilot schlug Befehle zur Initialisierung der Projektstruktur vor.

Vorgeschlagene Befehle:

- `flutter create visionquest` – Erstellung des Flutter-Frontends
- `npm init -y` – Initialisierung des Node.js Backends
- `npm install express cors dotenv` – Installation der benötigten Abhängigkeiten
- `mkdir -p Project/backend Project/frontend` – Erstellung der Ordnerstruktur

Ich habe die Befehle selbst ausgeführt und anschließend überprüft, ob das Projekt korrekt erstellt wurde und alle Abhängigkeiten funktionieren.

2. Error Handling für API-Responses

Copilot schlug ein Try-Catch-Pattern für HTTP-Requests und API-Responses vor, um Fehler strukturiert behandeln zu können.

Dieses Pattern habe ich als Standardstruktur für API-Antworten übernommen und anschließend bei allen Test-Endpunkten angewendet, damit sowohl erfolgreiche Antworten als auch Fehlerfälle einheitlich zurückgegeben werden.

3. Implementierung eines Test-Endpunkts

Copilot unterstützte bei der Erstellung eines einfachen Test-Endpunkts `/api/health`, der überprüft, ob das Backend korrekt läuft.

Die Route wurde von mir selbst in `server.js` implementiert. Anschließend wurde der Endpoint mit `curl http://localhost:5000/api/health` getestet und erfolgreich ausgeführt.

4. Fehlerbehebung während M1

Fehler 1 – CORS Error

Fehlermeldung: „Cross-Origin Request Blocked“

Ursache: Frontend und Backend liefen auf unterschiedlichen Ports.

Lösung: Einbindung der CORS-Middleware mit `app.use(cors())` in `server.js`. Danach funktionierten die Requests vom Flutter-Frontend.

Fehler 2 – Port-Konflikt

Fehlermeldung: „EADDRINUSE :::5000“

Ursache: Port 5000 wurde bereits von einem anderen Prozess verwendet.

Lösung: Der laufende Prozess wurde beendet und der Server anschließend erneut mit `npm start` gestartet.

Fehler 3 – JSON-Import in Flutter

Fehlermeldung: „Unknown method 'jsonDecode'“

Ursache: Fehlender Import der benötigten Dart-Bibliothek.

Lösung: Ergänzung des Imports `dart:convert` in `main.dart`, wodurch die JSON-Verarbeitung korrekt funktionierte.